

Learn Compiler Design Through xDSL: A Practical Course for AI/ML Engineers

Djordje Todorovic

August 2025

Contents

1	Preface: Who This Course Is For and Why Compilers Matter	3
1.1	Prerequisites and Intended Audience	3
1.2	Compilers: The Invisible Revolution That Changed Computing	3
1.3	The Core Mission: Represent and Transform	3
1.4	The Power of Serialize, Deserialize, and Verify	4
2	Introduction: Compilers as Representation and Transformation Engines	5
2.1	Welcome to the World of Compilers	5
2.2	Why xDSL?	5
2.3	Course Philosophy: Learn by Building	5
2.4	What You'll Learn	5
3	Chapter 1: Understanding Intermediate Representations (IR)	6
3.1	What is an IR?	6
3.2	The Compiler Pipeline: From Source to Machine Code	6
3.2.1	Frontend: Understanding Your Code	6
3.2.2	Middle-end: Optimization Central	7
3.2.3	Backend: Generating Machine Code	7
3.3	Learning from Giants: LLVM and MLIR	7
3.3.1	LLVM: The Industry Standard	7
3.3.2	MLIR: Multi-Level IR	7
3.4	Why xDSL Uses IR Instead of AST	8
3.5	SSA Form: Single Static Assignment	8
3.6	Your First xDSL Program	9
3.7	Understanding IR Structure	9
3.8	Dialects: Modular IR Design	9
4	Chapter 2: Operations and Operators	10
4.1	What is an Operation?	10
4.2	Creating Custom Operations	10
4.3	Operation Semantics	11
4.4	Attributes vs Operands	11
5	Chapter 3: Analysis Passes	11
5.1	What is Analysis?	11
5.2	Writing a Simple Analysis Pass	12
5.3	Use-Def Chains	12
5.4	Dominance Analysis	13

5.5	Dataflow Analysis	13
6	Chapter 4: Transformations and Optimizations	14
6.1	What is a Transformation?	14
6.2	Pattern-Based Rewriting	14
6.3	Common Optimizations	15
6.3.1	Dead Code Elimination	15
6.3.2	Common Subexpression Elimination	16
6.3.3	Loop Optimizations	17
6.4	Lowering Transformations	17
7	Chapter 5: Practical Examples	19
7.1	Building a Simple Expression Compiler	19
7.2	Implementing a Domain-Specific Optimization	20
7.3	Building a Mini ML Compiler	21
8	Chapter 6: Advanced Topics	22
8.1	Multi-Level IR and Progressive Lowering	22
8.2	Cost Models and Autotuning	23
8.3	Connection to ML Compilation	24
9	Chapter 7: Hands-On Exercises	24
9.1	Introduction: Learning by Doing	24
9.2	Exercise 1: Build Your Own Optimizer - Store-Load Forwarding	25
9.2.1	Context and Motivation	25
9.2.2	Testing Your Implementation	26
9.3	Exercise 2: Create a Custom Dialect - Quantum Computing	26
9.3.1	Context and Motivation	26
9.3.2	Challenge Extensions	28
9.4	Exercise 3: Implement Vectorization	28
9.4.1	Context and Motivation	28
9.4.2	Advanced Challenges	29
9.5	Exercise 4: Build a Simple JIT Compiler	29
9.5.1	Context and Motivation	29
9.5.2	Implementation Hints	31
10	Chapter 8: Real-World Applications	32
10.1	Case Study 1: Optimizing Neural Network Inference	32
10.2	Case Study 2: Domain-Specific Language Compiler	32
10.3	Case Study 3: Compiler for Distributed Computing	33
11	Project: Building a Matrix Operations Compiler	34
11.1	Project Overview	34
11.2	Phase 1: Frontend - Parsing the Input	35
11.3	Phase 2: Building the Parser	35
11.4	Phase 3: Designing the Matrix Dialect	36
11.5	Phase 5: Optimization - Double Transpose Elimination	38
11.6	Phase 6: The Complete Compiler Pipeline	39
11.7	Testing and Validation	40
11.8	Project Extensions and Exercises	40
11.9	Learning Outcomes and Key Takeaways	41
11.10	Conclusion	42

1 Preface: Who This Course Is For and Why Compilers Matter

1.1 Prerequisites and Intended Audience

This course is designed for engineers and developers who want to understand the fundamental concepts of compiler design through hands-on experience with xDSL. To get the most out of this material, you should have:

- **Python Proficiency:** Strong knowledge of Python is essential, as xDSL is a Python-native framework. You should be comfortable with classes, decorators, type hints, and the Python standard library.
- **Basic Programming Knowledge:** Understanding of fundamental programming concepts like functions, variables, control flow, data structures, and algorithms is required.
- **Machine Learning and AI Basics:** Familiarity with ML/AI concepts is crucial - you should understand tensors, neural networks, computational graphs, and optimization techniques. Experience with frameworks like PyTorch or TensorFlow will help you appreciate the compiler optimizations we'll explore.
- **Mathematical Foundations:** Basic understanding of linear algebra and discrete mathematics will be helpful, especially when working with optimization algorithms and graph transformations.

If you're an ML engineer curious about what happens "under the hood" when your models are compiled and optimized, or a systems programmer interested in building efficient code transformation tools, this course will bridge that gap using familiar Python syntax.

1.2 Compilers: The Invisible Revolution That Changed Computing

Compilers are perhaps the most transformative technology in the history of computing, yet they work so seamlessly that we rarely think about them. To understand their revolutionary impact, consider this: the Linux kernel, which powers billions of devices worldwide, was initially written in assembly language - a tedious, error-prone process where programmers had to think in terms of individual CPU instructions and memory addresses. Each line of assembly code corresponded directly to a machine instruction, making even simple tasks require hundreds of lines of code.

Then came the evolution of the C compiler. As compilers matured, they could translate high-level C code into assembly with equal or sometimes even better performance than hand-written assembly. This transformation was revolutionary - Linux could be rewritten in C, making it portable across different architectures, easier to maintain, and accessible to a broader community of developers. What once required intimate knowledge of specific CPU architectures could now be expressed in readable, maintainable code. The compiler handled the complex translation, optimization, and architecture-specific details automatically.

This same revolution continues today in machine learning. Just as C compilers freed developers from assembly, ML compilers like XLA, TVM, and MLIR free ML engineers from writing architecture-specific kernels. Your PyTorch model can run efficiently on CPUs, GPUs, TPUs, or custom accelerators, all thanks to sophisticated compiler technology working behind the scenes.

1.3 The Core Mission: Represent and Transform

At its heart, a compiler's job is elegantly simple yet profoundly powerful: to **represent** programs in structured forms and to **transform** these representations to achieve specific goals. Think of a compiler as a sophisticated translator that not only converts between languages but also understands the meaning of what it's translating deeply enough to improve it along the way.

This representation and transformation paradigm involves:

- **Representation:** Converting source code into structured data (Abstract Syntax Trees, Intermediate Representations, Control Flow Graphs) that capture the program’s semantics precisely
- **Analysis:** Understanding data dependencies, control flow, memory access patterns, and optimization opportunities within these representations
- **Transformation:** Systematically modifying these representations to optimize performance, reduce resource usage, or target specific hardware architectures
- **Preservation:** Ensuring that transformations maintain the original program’s correctness and semantics

1.4 The Power of Serialize, Deserialize, and Verify

A crucial but often overlooked aspect of compiler infrastructure is the ability to serialize, deserialize, and verify intermediate representations. This capability fundamentally changes how we can work with compilers and is a cornerstone of modern compiler design.

Why These Capabilities Matter:

- **Serialization** allows us to save the compiler’s intermediate state to disk, enabling separate compilation, caching of optimization results, and distribution of compiled modules. You can pause compilation, save the IR, and resume later - or on a different machine entirely.
- **Deserialization** lets us load previously compiled modules, compose them together, and build large systems incrementally. This is essential for modular compilation and linking.
- **Verification** ensures that the IR is well-formed and obeys the type system and semantic rules. This catches errors early, enables safe transformations, and provides guarantees about program behavior. Without verification, compiler bugs could silently corrupt programs.

The xDSL infrastructure we’re about to explore has these capabilities built into its core. Every operation, every transformation, and every intermediate state can be serialized to a human-readable textual format, loaded back, and verified for correctness. This isn’t just a convenience feature - it’s fundamental to building robust, composable compiler pipelines. You can inspect the IR at any stage, debug transformations by examining the before-and-after states, and even hand-write IR for testing.

This approach contrasts sharply with traditional compilers where the intermediate states are often opaque binary structures locked inside the compiler’s memory. With xDSL (inspired by MLIR), the entire compilation pipeline becomes transparent, debuggable, and extensible. You can serialize the IR after each optimization pass, analyze what changed, and even replay specific transformations. This visibility and control make compiler development more like software engineering and less like black magic.

As you progress through this course, you’ll come to appreciate how these fundamental capabilities - serialize, deserialize, and verify - enable you to build reliable, maintainable, and powerful compilation tools. They transform the compiler from a monolithic black box into a modular, inspectable, and trustworthy system.

2 Introduction: Compilers as Representation and Transformation Engines

2.1 Welcome to the World of Compilers

If you're coming from an AI/ML background, you already understand the power of transforming data through layers of abstraction. Compilers are remarkably similar: they take programs written in one representation and systematically transform them into another, more optimized or more executable form.

Think of a compiler as a sophisticated pipeline that:

1. **Represents** programs as structured data (Abstract Syntax Trees, Intermediate Representations)
2. **Transforms** these representations through a series of optimization passes
3. **Analyzes** code to understand dependencies, patterns, and optimization opportunities
4. **Generates** efficient target code for specific hardware

2.2 Why xDSL?

xDSL is a Python-native compiler framework that makes compiler concepts accessible to Python programmers. Unlike traditional compiler frameworks written in C++ (like LLVM), xDSL allows you to:

- Build compilers using familiar Python syntax
- Prototype and experiment quickly
- Understand compiler internals without low-level complexity
- Leverage the entire Python ecosystem for analysis and visualization

2.3 Course Philosophy: Learn by Building

This course takes a hands-on approach. Instead of starting with theory, we'll build small compilers and optimization passes from day one. Each concept will be introduced through practical examples that you can run, modify, and experiment with.

2.4 What You'll Learn

By the end of this course, you'll understand:

- How compilers represent programs internally (IR - Intermediate Representations)
- What operations and operators mean in a compiler context
- How to write analysis passes that extract information from code
- How to implement transformations that optimize programs
- The connection between compiler optimizations and ML model optimization

3 Chapter 1: Understanding Intermediate Representations (IR)

3.1 What is an IR?

An Intermediate Representation (IR) is the compiler's internal language for representing programs. Just as neural networks use tensors to represent data, compilers use IRs to represent code structure and semantics.

Key properties of IRs:

- **Abstract:** Higher-level than machine code, but more structured than source code
- **Explicit:** Makes implicit operations visible (e.g., memory allocations, type conversions)
- **Analyzable:** Designed for easy pattern matching and transformation
- **Hierarchical:** Can represent nested structures (functions, loops, blocks)

3.2 The Compiler Pipeline: From Source to Machine Code

Before diving into xDSL specifics, let's understand how compilers work. A compiler is traditionally divided into three main phases, each with specific responsibilities:

3.2.1 Frontend: Understanding Your Code

The frontend translates source code into an intermediate representation. It consists of:

1. Lexical Analysis (Lexer/Tokenizer)

- Breaks source code into tokens (keywords, identifiers, operators)
- Like splitting a sentence into words
- Example: "x = 42 + y" becomes tokens: [ID:x, ASSIGN, NUM:42, PLUS, ID:y]

2. Syntax Analysis (Parser)

- Builds an Abstract Syntax Tree (AST) from tokens
- Checks grammatical structure
- Like diagramming a sentence to understand its structure

```
1 # Source: x = 42 + y
2 # AST representation:
3 Assignment(
4     left=Variable("x"),
5     right=BinaryOp(
6         op="+",
7         left=Number(42),
8         right=Variable("y")
9     )
10 )
```

Listing 1: Simple AST Example

3. Semantic Analysis

- Type checking: Is this operation valid?
- Symbol resolution: What does this variable refer to?
- Example: Can't add a string to a number, variables must be declared

3.2.2 Middle-end: Optimization Central

The middle-end works on the IR to optimize code:

- **Platform-independent:** Optimizations work for any target
- **Examples:** Dead code elimination, constant folding, loop optimization
- **Multiple passes:** Each optimization is a separate pass over the IR

3.2.3 Backend: Generating Machine Code

The backend translates optimized IR to target-specific code:

- **Instruction selection:** Choose machine instructions
- **Register allocation:** Assign variables to CPU registers
- **Instruction scheduling:** Order operations for performance

3.3 Learning from Giants: LLVM and MLIR

3.3.1 LLVM: The Industry Standard

LLVM (Low Level Virtual Machine) is the most widely used compiler infrastructure:

- Powers Clang (C/C++), Swift, Rust, and many other languages
- Provides a well-defined IR (LLVM IR) that many frontends target
- Excellent tutorial: Kaleidoscope Tutorial
- Shows how to build a complete compiler for a simple language

```
1 ; LLVM IR for: int add(int a, int b) { return a + b; }
2 define i32 @add(i32 %a, i32 %b) {
3   entry:
4     %sum = add i32 %a, %b
5     ret i32 %sum
6 }
```

Listing 2: LLVM IR Example

3.3.2 MLIR: Multi-Level IR

MLIR (Multi-Level Intermediate Representation) extends LLVM's concepts:

- Developed by Google, now part of LLVM project
- Supports multiple IRs (dialects) in the same infrastructure
- Used by TensorFlow, PyTorch, and other ML frameworks
- Tutorial: Toy Language Tutorial
- xDSL is heavily inspired by MLIR but implemented in Python

```

1 // High-level ML operation
2 %result = "tf.MatMul"(%a, %b) : (tensor<2x3xf32>, tensor<3x4xf32>)
3                                     -> tensor<2x4xf32>
4
5 // After lowering to linalg dialect
6 %result = linalg.matmul ins(%a, %b : memref<2x3xf32>, memref<3x4xf32>)
7                                     outs(%c : memref<2x4xf32>)
8
9 // After lowering to loops
10 scf.for %i = 0 to 2 {
11   scf.for %j = 0 to 4 {
12     scf.for %k = 0 to 3 {
13       // actual computation
14     }
15   }
16 }

```

Listing 3: MLIR Example - Multiple Abstraction Levels

3.4 Why xDSL Uses IR Instead of AST

While traditional compilers start with AST, xDSL (like MLIR) works directly with IR because:

- **Uniformity:** All transformations work on the same structure
- **Composability:** Easy to combine different languages/dialects
- **Reusability:** Optimizations work across different source languages
- **Analysis-friendly:** SSA form makes many analyses trivial

Think of it this way:

- **AST:** Like a sentence diagram - shows structure
- **IR:** Like assembly with types - shows computation

3.5 SSA Form: Single Static Assignment

xDSL uses SSA (Single Static Assignment) form, where each variable is assigned exactly once. This is similar to functional programming and makes many analyses simpler.

```

1 # Traditional form
2 x = 5
3 x = x + 1
4 y = x * 2
5
6 # SSA form (conceptual)
7 x_0 = 5
8 x_1 = x_0 + 1
9 y_0 = x_1 * 2

```

Listing 4: Traditional vs SSA Form

Why SSA matters:

- **No ambiguity:** Each value has exactly one definition
- **Easy analysis:** Use-def chains are explicit
- **Better optimization:** Many optimizations become simpler
- **Parallel-friendly:** Dependencies are clear

3.6 Your First xDSL Program

Note: all code examples in this document have been updated for xDSL version 0.71.0. If an example does not work when you read this, check the current function and class signatures, as the xDSL API changes between releases.

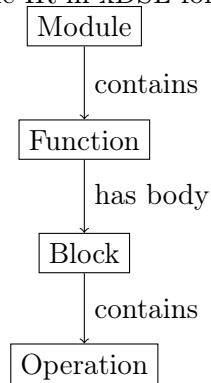
Let's create a simple IR program using xDSL:

```
1 from xdsl.dialects import arith, func
2 from xdsl.dialects.builtin import i32, FunctionType, ModuleOp
3 from xdsl.builder import ImplicitBuilder
4
5 # 1) an empty module
6 module = ModuleOp([])
7
8 # 2) open the module body and build inside it
9 with ImplicitBuilder(module.body):
10     # Create function signature: (i32, i32) -> i32
11     func_type = FunctionType.from_lists([i32, i32], [i32])
12     add = func.FuncOp("add", func_type)
13
14     # 3) open the function body; 'as (...)' yields the block arguments
15     with ImplicitBuilder(add.body) as (arg0, arg1):
16         # Create add operation
17         result = arith.AddiOp(arg0, arg1)
18         # Return the result
19         func.ReturnOp(result)
20
21 print(module)
```

Listing 5: Creating IR in xDSL

3.7 Understanding IR Structure

The IR in xDSL follows a hierarchical structure:



3.8 Dialects: Modular IR Design

xDSL organizes operations into *dialects* - collections of related operations and types. This is similar to how PyTorch has different modules (nn, optim, etc.).

Common dialects in xDSL:

- **arith**: Arithmetic operations (add, multiply, divide)
- **func**: Function definitions and calls
- **memref**: Memory operations (like NumPy arrays)

- `scf`: Structured control flow (loops, conditionals)
- `linalg`: Linear algebra operations (matrix multiply, convolution)

4 Chapter 2: Operations and Operators

4.1 What is an Operation?

An operation in xDSL is the fundamental unit of computation. Each operation:

- Has a name (e.g., `arith.addi` for integer addition)
- Takes input values (operands)
- Produces output values (results)
- May have attributes (compile-time constants)
- Can contain regions (nested blocks of operations)

4.2 Creating Custom Operations

Let's define a custom operation for matrix operations:

```

1 from xdsl.ir import Dialect, SSAValue
2 from xdsl.irdl import irdl_op_definition, IRDLOperation, operand_def,
   result_def
3 from xdsl.dialects.builtin import TensorType
4 from xdsl.utils.exceptions import VerifyException
5
6 def infer_matmul_type(lhs: SSAValue, rhs: SSAValue) -> TensorType:
7     m, _ = lhs.type.get_shape()          # (M, K)
8     _, n = rhs.type.get_shape()          # (K, N)
9     return TensorType(lhs.type.element_type, [m, n])    # -> (M, N)
10
11 @irdl_op_definition
12 class MatMulOp(IRDLOperation):
13     """Matrix multiplication operation"""
14     name = "my_dialect.matmul"
15
16     # Define operands (inputs)
17     lhs = operand_def(TensorType)
18     rhs = operand_def(TensorType)
19
20     # Define results (outputs)
21     result = result_def(TensorType)
22
23     def __init__(self, lhs: SSAValue, rhs: SSAValue):
24         super().__init__(operands=[lhs, rhs],
25                          result_types=[infer_matmul_type(lhs, rhs)])
26
27     def verify_(self):
28         """Verify that the operation is well-formed"""
29         # Check that dimensions are compatible
30         lhs_shape = self.lhs.type.get_shape()
31         rhs_shape = self.rhs.type.get_shape()
32
33         if lhs_shape[-1] != rhs_shape[0]:

```

```
34         raise VerifyException("Incompatible matrix dimensions")
```

Listing 6: Defining Custom Operations

4.3 Operation Semantics

Every operation has well-defined semantics - what it means to execute that operation:

```
1 # Arithmetic operations
2 add_op = arith.AddiOp(a, b)           # result = a + b
3 mul_op = arith.MuliOp(a, b)           # result = a * b
4
5 # Memory operations
6 alloc_op = memref.AllocOp([], [], MemRefType(f32, [10, 20])) #
    allocate 10x20 array
7 load_op = memref.LoadOp.get(memref_value, indices)           # load
    from memory
8 store_op = memref.StoreOp.get(value, memref_value, indices)  # store to
    memory
9
10 # Control flow operations
11 for_op = scf.ForOp(lower_bound, upper_bound, step, [])       # for loop
12 if_op = scf.IfOp(condition, [result_types], true_region)     # if-then-
    else
```

Listing 7: Operation Semantics Example

4.4 Attributes vs Operands

Understanding the difference between attributes and operands is crucial:

- **Attributes:** Compile-time constants (shapes, types, flags)
- **Operands:** Runtime values (variables, intermediate results)

```
1 # Constant has an attribute (the value)
2 const = arith.ConstantOp.from_int_and_width(42, 32) # 42 is an
    attribute
3
4 # Add has operands (runtime values)
5 result = arith.AddiOp(x, y) # x and y are operands
6
7 # Alloc has attributes (shape) but no data operands
8 mem = memref.AllocOp([], [], MemRefType(f32, [100])) # shape is
    attribute
```

Listing 8: Attributes vs Operands

5 Chapter 3: Analysis Passes

5.1 What is Analysis?

Analysis passes extract information from IR without modifying it. They answer questions like:

- Which variables are used where? (Use-Def Analysis)
- Which operations can run in parallel? (Dependency Analysis)
- What are the loop bounds? (Range Analysis)
- Which operations are dead code? (Liveness Analysis)

5.2 Writing a Simple Analysis Pass

Let's write an analysis that counts operations by type:

```
1 from xdsl.context import Context
2 from xdsl.passes import ModulePass
3 from xdsl.dialects import builtin
4 from collections import defaultdict
5
6 class OperationCounterPass(ModulePass):
7     """Count operations by type in the IR"""
8
9     name = "count-ops"
10
11     def apply(self, ctx: Context, module: builtin.ModuleOp) -> None:
12         op_counts = defaultdict(int)
13
14         # Walk through all operations in the module
15         for op in module.walk():
16             op_counts[op.name] += 1
17
18         # Print analysis results
19         print("Operation counts:")
20         for op_name, count in sorted(op_counts.items()):
21             print(f"  {op_name}: {count}")
22
23         # Store results for use by other passes
24         self.op_counts = op_counts
```

Listing 9: Operation Counter Analysis

5.3 Use-Def Chains

Use-Def chains track where values are defined and used:

```
1 from xdsl.context import Context
2 from xdsl.passes import ModulePass
3 from xdsl.dialects import builtin
4 from xdsl.ir import OpResult
5
6 class UseDefAnalysis(ModulePass):
7     """Analyze use-def relationships"""
8
9     name = "use-def"
10
11     def apply(self, ctx: Context, module: builtin.ModuleOp) -> None:
12         # Build use-def chains
13         for op in module.walk():
14             for result in op.results:
15                 print(f"Value {result} defined by {op.name}")
16                 print(f"  Used by: {[use.operation.name for use in
17 result.uses]}")
18
19     def find_unused_values(self, module: builtin.ModuleOp) -> list[
20 OpResult]:
21         """Find values that are never used"""
22         unused = []
23         for op in module.walk():
24             for result in op.results:
```

```

23         if not result.uses:
24             unused.append(result)
25     return unused

```

Listing 10: Use-Def Analysis

5.4 Dominance Analysis

Dominance tells us which operations must execute before others:

```

1 from xdsl.context import Context
2 from xdsl.passes import ModulePass
3 from xdsl.dialects import builtin, func
4 from xdsl.irdl.dominance import DominanceInfo
5
6 class DominanceAnalysis(ModulePass):
7     """Analyze control flow dominance."""
8
9     name = "dominance"
10
11     def apply(self, ctx: Context, module: builtin.ModuleOp) -> None:
12         # For each function in the module
13         for func_op in module.body.ops:
14             if isinstance(func_op, func.FuncOp):
15                 # DominanceInfo works on a region and answers block-
16                 # level
17                 # dominance queries: dominates(a, b) /
18                 # strictly_dominates(a, b).
19                 dom = DominanceInfo(func_op.body)
20                 blocks = list(func_op.body.blocks)
21                 names = {b: f"bb{i}" for i, b in enumerate(blocks)}
22
23                 # For each block, list the blocks that dominate it
24                 for b in blocks:
25                     dominators = [names[a] for a in blocks if dom.
26                                 dominates(a, b)]
27                     print(f"Block {names[b]} dominated by: {dominators}")
28
29     """

```

Listing 11: Dominance Analysis

5.5 Dataflow Analysis

Dataflow analysis propagates information through the program:

```

1 from xdsl.context import Context
2 from xdsl.passes import ModulePass
3 from xdsl.dialects import builtin, arith
4
5 class ConstantPropagationAnalysis(ModulePass):
6     """Analyze which values are compile-time constants"""
7
8     name = "const-prop"
9
10    def apply(self, ctx: Context, module: builtin.ModuleOp) -> None:
11        known_constants = {}
12
13        # First pass: identify constants

```

```

14     # (op.result is used as an SSA-value key; op.value.value.data
    is the int)
15     for op in module.walk():
16         if isinstance(op, arith.ConstantOp):
17             known_constants[op.result] = op.value.value.data
18
19     # Iteratively propagate constants
20     changed = True
21     while changed:
22         changed = False
23         for op in module.walk():
24             if isinstance(op, arith.AddiOp):
25                 # If both operands are known constants
26                 if (op.lhs in known_constants and
27                     op.rhs in known_constants):
28                     result_value = (known_constants[op.lhs] +
29                                     known_constants[op.rhs])
30                     if op.result not in known_constants:
31                         known_constants[op.result] = result_value
32                         changed = True
33
34     return known_constants

```

Listing 12: Constant Propagation Analysis

6 Chapter 4: Transformations and Optimizations

6.1 What is a Transformation?

Transformations modify the IR to improve performance, reduce code size, or prepare for further optimizations. Unlike analysis passes, transformations actually change the program structure.

Key principles:

- **Correctness:** Must preserve program semantics
- **Profitability:** Should improve some metric (speed, size, power)
- **Composability:** Should work well with other transformations

6.2 Pattern-Based Rewriting

xDSL provides powerful pattern matching for transformations:

```

1 from xdsl.dialects import arith
2 from xdsl.pattern_rewriter import (
3     PatternRewriter,
4     RewritePattern,
5     op_type_rewrite_pattern,
6 )
7
8 class FoldAddZero(RewritePattern):
9     """Fold x + 0 -> x"""
10
11     @op_type_rewrite_pattern
12     def match_and_rewrite(self, op: arith.AddiOp,
13                           rewriter: PatternRewriter) -> None:
14         # Check if right operand is zero
15         if isinstance(op.rhs.owner, arith.ConstantOp):

```

```

16         if op.rhs.owner.value.value.data == 0:
17             # Replace all uses of the add with the left operand
18             rewriter.replace_all_uses_with(op.result, op.lhs)
19             rewriter.erase(op)
20             return
21
22     # Check if left operand is zero
23     if isinstance(op.lhs.owner, arith.ConstantOp):
24         if op.lhs.owner.value.value.data == 0:
25             rewriter.replace_all_uses_with(op.result, op.rhs)
26             rewriter.erase(op)

```

Listing 13: Simple Algebraic Optimization

6.3 Common Optimizations

6.3.1 Dead Code Elimination

Remove operations whose results are never used:

```

1 from xdsl.context import Context
2 from xdsl.passes import ModulePass
3 from xdsl.dialects import builtin, memref, func
4 from xdsl.ir import Operation
5
6 class DeadCodeElimination(ModulePass):
7     """Remove unused operations"""
8
9     name = "my-dce"
10
11     def apply(self, ctx: Context, module: builtin.ModuleOp) -> None:
12         # Iterate to a fixpoint: removing a dead op can make its
13         # operands dead.
14         changed = True
15         while changed:
16             changed = False
17             ops_to_remove = []
18
19             for op in module.walk():
20                 # Skip operations with side effects
21                 if self.has_side_effects(op):
22                     continue
23
24                 # IMPORTANT: only consider ops that actually PRODUCE a
25                 # value.
26                 # all(...) over an empty result list is True, so
27                 # without this
28                 # guard we would wrongly delete result-less ops such as
29                 # func.return, func.func and the module itself.
30                 if not op.results:
31                     continue
32
33                 # Dead if none of its results are used
34                 if all(not result.uses for result in op.results):
35                     ops_to_remove.append(op)
36
37             # Remove dead operations
38             for op in ops_to_remove:
39                 op.detach()

```

```

37         op.erase()
38         changed = True
39
40     def has_side_effects(self, op: Operation) -> bool:
41         """Check if operation has side effects"""
42         # Memory writes, function calls, etc.
43         return isinstance(op, (memref.StoreOp, func.CallOp))

```

Listing 14: Dead Code Elimination

6.3.2 Common Subexpression Elimination

Identify and eliminate redundant computations:

```

1  from xdsl.context import Context
2  from xdsl.passes import ModulePass
3  from xdsl.dialects import builtin
4  from xdsl.ir import Operation
5  from xdsl.traits import Pure
6
7  class CommonSubexpressionElimination(ModulePass):
8      """Eliminate redundant computations"""
9
10     name = "my-cse"
11
12     def apply(self, ctx: Context, module: builtin.ModuleOp) -> None:
13         # Map from expression key to the SSA value that first produced
14         it.
15         expression_map = {}
16
17         for op in list(module.walk()):
18             if self.is_pure(op):
19                 # Key for this expression
20                 key = self.get_expression_key(op)
21
22                 if key in expression_map:
23                     # Duplicate found: reuse the earlier result, drop
24                     this op.
25                     # NB: in straight-line code the first occurrence
26                     dominates
27                     # later ones; a general CSE would check dominance
28                     here.
29                     op.results[0].replace_all_uses_with(expression_map[
30                     key])
31                     op.detach()
32                     op.erase()
33                 else:
34                     expression_map[key] = op.results[0]
35
36     def is_pure(self, op: Operation) -> bool:
37         # Only fold value-producing ops with no side effects.
38         return bool(op.results) and op.has_trait(Pure)
39
40     def get_expression_key(self, op: Operation):
41         """Create a hashable key for the expression.
42
43         Must include attributes AND properties, not just operands:
44         otherwise

```



```

39         two different constants (e.g. 'constant 5' and 'constant 7',
both with
40         no operands) would collide and be wrongly merged.
41         """
42         return (op.name,
43                 tuple(op.operands),
44                 tuple(sorted(op.attributes.items()))),
45                 tuple(sorted(op.properties.items())))

```

Listing 15: Common Subexpression Elimination

6.3.3 Loop Optimizations

Optimize loop structures for better performance:

```

1 from xdsl.context import Context
2 from xdsl.passes import ModulePass
3 from xdsl.dialects import builtin, scf, arith
4
5 class LoopUnrolling(ModulePass):
6     """Unroll small loops"""
7
8     name = "loop-unroll"
9
10    def apply(self, ctx: Context, module: builtin.ModuleOp) -> None:
11        for op in module.walk():
12            if isinstance(op, scf.ForOp):
13                if self.should_unroll(op):
14                    self.unroll_loop(op)
15
16    def should_unroll(self, loop: scf.ForOp) -> bool:
17        """Decide if loop should be unrolled"""
18        # Get loop bounds if constant
19        if (isinstance(loop.lb.owner, arith.ConstantOp) and
20            isinstance(loop.ub.owner, arith.ConstantOp)):
21            lb = loop.lb.owner.value.value.data
22            ub = loop.ub.owner.value.value.data
23            iterations = ub - lb
24
25            # Unroll small loops
26            return iterations <= 4
27        return False
28
29    def unroll_loop(self, loop: scf.ForOp):
30        """Completely unroll a loop"""
31        # Clone loop body for each iteration
32        # Update induction variable references
33        # Remove original loop
34        pass # Implementation details omitted

```

Listing 16: Loop Unrolling

6.4 Lowering Transformations

Lowering transforms high-level operations into simpler ones:

```

1 from xdsl.context import Context
2 from xdsl.passes import ModulePass
3 from xdsl.dialects import builtin, arith, memref, scf

```

```

4 from xdsl.dialects.builtin import (MemRefType, f32, IndexType,
5   IntegerAttr, FloatAttr)
6 from xdsl.builder import ImplicitBuilder, Builder, InsertPoint
7
8 class LowerMatrixOps(ModulePass):
9     """Lower matrix operations to loops.
10
11     Assumes a memref-based MatMulOp: (MxK) @ (KxN) -> (MxN).
12     """
13
14     name = "lower-matmul"
15
16     def apply(self, ctx: Context, module: builtin.ModuleOp) -> None:
17         for op in list(module.walk()):
18             if isinstance(op, MatMulOp):
19                 self.lower_matmul(op)
20
21     def lower_matmul(self, matmul: MatMulOp):
22         """Lower matmul to three nested loops."""
23         A, B = matmul.lhs, matmul.rhs
24
25         # Get dimensions
26         m, k = A.type.get_shape()
27         _, n = B.type.get_shape()
28         idx = IndexType()
29
30         # Build the loop nest just before the matmul op
31         with ImplicitBuilder(Builder(InsertPoint.before(matmul))):
32             c0 = arith.ConstantOp(IntegerAttr(0, idx))
33             c1 = arith.ConstantOp(IntegerAttr(1, idx))
34             cM = arith.ConstantOp(IntegerAttr(m, idx))
35             cN = arith.ConstantOp(IntegerAttr(n, idx))
36             cK = arith.ConstantOp(IntegerAttr(k, idx))
37             zero = arith.ConstantOp(FloatAttr(0.0, f32))
38
39             # Allocate result
40             C = memref.AllocOp.get(matmul.result.type.element_type,
41                                   shape=[m, n])
42
43             # Outer loop over i
44             @Builder.implicit_region((idx,))
45             def i_body(i_args):
46                 (i,) = i_args
47
48                 # Middle loop over j
49                 @Builder.implicit_region((idx,))
50                 def j_body(j_args):
51                     (j,) = j_args
52
53                     # Inner loop over k with an accumulator (iter_arg)
54                     @Builder.implicit_region((idx, f32))
55                     def k_body(k_args):
56                         (kk, acc) = k_args
57                         a_elem = memref.LoadOp.get(A, [i, kk])
58                         b_elem = memref.LoadOp.get(B, [kk, j])
59                         prod = arith.MulfOp(a_elem, b_elem)
60                         new_acc = arith.AddfOp(acc, prod)
61                         scf.YieldOp(new_acc)

```

```

62         k_loop = scf.ForOp(c0, cK, c1, [zero], k_body)
63         # Store the accumulated result
64         memref.StoreOp.get(k_loop.results[0], C, [i, j])
65         scf.YieldOp()
66
67     scf.ForOp(c0, cN, c1, [], j_body)
68     scf.YieldOp()
69
70     scf.ForOp(c0, cM, c1, [], i_body)
71
72     # Replace matmul with the lowered result buffer, then remove it
73     matmul.result.replace_all_uses_with(C.results[0])
74     matmul.detach()
75     matmul.erase()
76

```

Listing 17: Lowering High-Level Operations

7 Chapter 5: Practical Examples

7.1 Building a Simple Expression Compiler

Let's build a complete compiler for arithmetic expressions:

```

1 from dataclasses import dataclass
2
3 from xdsl.dialects import arith, func
4 from xdsl.dialects.builtin import i32, FunctionType, ModuleOp
5 from xdsl.ir import SSAValue
6
7 # Define AST nodes
8 @dataclass
9 class Expr:
10     pass
11
12 @dataclass
13 class Number(Expr):
14     value: int
15
16 @dataclass
17 class BinaryOp(Expr):
18     left: Expr
19     op: str
20     right: Expr
21
22 # Parser (simplified)
23 def parse(text: str) -> Expr:
24     """Parse expression like '2 + 3 * 4'"""
25     # Implementation omitted for brevity
26     pass
27
28 # IR Generator
29 class ExprIRGen:
30     def compile(self, expr: Expr) -> SSAValue:
31         """Compile expression to xDSL IR"""
32         if isinstance(expr, Number):
33             return arith.ConstantOp.from_int_and_width(
34                 expr.value, 32).result

```

```

35
36     elif isinstance(expr, BinaryOp):
37         left = self.compile(expr.left)
38         right = self.compile(expr.right)
39
40         if expr.op == '+':
41             return arith.AddiOp(left, right).result
42         elif expr.op == '*':
43             return arith.MuliOp(left, right).result
44         # ... other operations
45
46 # Optimizer
47 def optimize(module: ModuleOp):
48     """Apply optimization passes"""
49     passes = [
50         ConstantFolding(),
51         CommonSubexpressionElimination(),
52         DeadCodeElimination(),
53     ]
54
55     for pass_instance in passes:
56         pass_instance.apply(module)
57
58 # Complete pipeline
59 def compile_expression(text: str):
60     # Parse
61     ast = parse(text)
62
63     # Generate IR
64     ir_gen = ExprIRGen()
65     result = ir_gen.compile(ast)
66
67     # Wrap in module
68     module = ModuleOp([
69         func.FuncOp("compute",
70                     FunctionType.from_lists([], [i32]),
71                     body=[func.ReturnOp(result)])
72     ])
73
74     # Optimize
75     optimize(module)
76
77     return module

```

Listing 18: Expression Compiler

7.2 Implementing a Domain-Specific Optimization

Let's create an optimization for neural network operations:

```

1 class FuseBatchNormIntoConv(ModulePass):
2     """Fuse batch normalization into preceding convolution"""
3
4     name = "fuse-bn-conv"
5
6     def apply(self, ctx, module):
7         for conv in module.walk():
8             if isinstance(conv, nn.Conv2d):

```

```

9         # Look for batch norm following conv
10        for use in conv.result.uses:
11            if isinstance(use.operation, nn.BatchNorm2d):
12                self.fuse(conv, use.operation)
13
14    def fuse(self, conv, bn):
15        """Fuse batch norm parameters into convolution"""
16        # Extract batch norm parameters
17        gamma = bn.gamma          # scale
18        beta = bn.beta            # shift
19        mean = bn.running_mean
20        var = bn.running_var
21        eps = bn.eps
22
23        # Compute fused weights and bias
24        std = sqrt(var + eps)
25        fused_weight = conv.weight * (gamma / std)
26        fused_bias = gamma * (conv.bias - mean) / std + beta
27
28        # Create new convolution with fused parameters
29        fused_conv = nn.Conv2d(
30            weight=fused_weight,
31            bias=fused_bias,
32            # ... other conv parameters
33        )
34
35        # Replace conv -> bn with fused conv
36        bn.result.replace_all_uses_with(fused_conv.result)
37        conv.erase()
38        bn.erase()

```

Listing 19: Neural Network Optimization

7.3 Building a Mini ML Compiler

Let's create a compiler for a subset of ML operations:

```

1  # Illustrative: defines an invented 'ml' dialect to sketch fusion +
   lowering.
2  from xdsl.ir import Dialect
3  from xdsl.irdl import irdl_op_definition, IRDLOperation, operand_def,
   result_def
4  from xdsl.dialects.builtin import TensorType
5  from xdsl.pattern_rewriter import (
6      RewritePattern, PatternRewriter, op_type_rewrite_pattern,
7  )
8
9  @irdl_op_definition
10 class LinearOp(IRDLOperation):
11     """Linear layer:  $y = Wx + b$ """
12     name = "ml.linear"
13
14     input = operand_def(TensorType)
15     weight = operand_def(TensorType)
16     bias = operand_def(TensorType)
17
18     output = result_def(TensorType)
19

```

```

20 @irdl_op_definition
21 class ReluOp(IRDLOperation):
22     """ReLU activation"""
23     name = "ml.relu"
24
25     input = operand_def(TensorType)
26     output = result_def(TensorType)
27
28 MLDialect = Dialect("ml", [LinearOp, ReluOp])
29
30 # Optimization: Fuse Linear + ReLU
31 class FuseLinearRelu(RewritePattern):
32     @op_type_rewrite_pattern
33     def match_and_rewrite(self, relu: ReluOp,
34                           rewriter: PatternRewriter):
35         # Check if input is from a linear layer
36         if isinstance(relu.input.owner, LinearOp):
37             linear = relu.input.owner
38
39             # Create fused operation
40             fused = LinearReluOp(
41                 linear.input,
42                 linear.weight,
43                 linear.bias,
44             )
45
46             # Replace both ops with the fused version
47             rewriter.replace_all_uses_with(relu.output, fused.output)
48             rewriter.erase(linear)
49
50 # Lower to hardware operations
51 class LowerMLToHardware(ModulePass):
52     """Lower ML ops to target hardware"""
53
54     name = "lower-ml"
55
56     def apply(self, ctx, module):
57         target = self.get_target() # CPU, GPU, TPU, etc.
58
59         for op in module.walk():
60             if isinstance(op, LinearOp):
61                 if target == "gpu":
62                     self.lower_linear_to_cublas(op)
63                 elif target == "cpu":
64                     self.lower_linear_to_loops(op)

```

Listing 20: Mini ML Compiler

8 Chapter 6: Advanced Topics

8.1 Multi-Level IR and Progressive Lowering

Real compilers use multiple levels of abstraction:

```

1 class CompilationPipeline:
2     """Progressive lowering through multiple IR levels"""
3
4     def compile(self, source_code: str) -> str:

```

```

5      # Level 1: High-level ML operations
6      ml_ir = parse_to_ml_dialect(source_code)
7      ml_ir = optimize_ml_level(ml_ir)
8
9      # Level 2: Linear algebra operations
10     linalg_ir = lower_ml_to_linalg(ml_ir)
11     linalg_ir = optimize_linalg_level(linalg_ir)
12
13     # Level 3: Loops and memory operations
14     loop_ir = lower_linalg_to_loops(linalg_ir)
15     loop_ir = optimize_loops(loop_ir)
16
17     # Level 4: Target-specific operations
18     target_ir = lower_to_target(loop_ir)
19     target_ir = optimize_target_specific(target_ir)
20
21     # Code generation
22     return generate_code(target_ir)

```

Listing 21: Multi-Level Compilation Pipeline

8.2 Cost Models and Autotuning

Choosing the best optimization requires understanding costs:

```

1  class CostModel:
2      """Estimate operation costs"""
3
4      def estimate_cost(self, op) -> float:
5          if isinstance(op, arith.MulOp):
6              return 1.0 # FP multiply cost
7          elif isinstance(op, memref.LoadOp):
8              return self.memory_latency(op)
9          elif isinstance(op, scf.ForOp):
10             iterations = self.analyze_trip_count(op)
11             body_cost = sum(self.estimate_cost(body_op)
12                             for body_op in op.body.walk())
13             return iterations * body_cost
14
15  class AutotuningPass(ModulePass):
16      """Try different optimization strategies"""
17
18      name = "autotune"
19
20      def apply(self, ctx, module):
21          strategies = [
22              self.try_vectorization,
23              self.try_loop_tiling,
24              self.try_parallelization,
25          ]
26
27          best_module = module
28          best_cost = self.cost_model.estimate_cost(module)
29
30          for strategy in strategies:
31              candidate = strategy(module.clone())
32              cost = self.cost_model.estimate_cost(candidate)
33

```

```

34         if cost < best_cost:
35             best_module = candidate
36             best_cost = cost
37
38     return best_module

```

Listing 22: Cost-Based Optimization

8.3 Connection to ML Compilation

Modern ML frameworks are essentially compilers:

- **PyTorch JIT**: Traces Python code to TorchScript IR
- **TensorFlow XLA**: Compiles TF graphs to optimized kernels
- **JAX**: Uses XLA for JIT compilation
- **Apache TVM**: Optimizes ML models for various hardware

The concepts you’ve learned apply directly:

- **Graph optimization** = IR transformation
- **Kernel fusion** = Operation merging
- **Memory planning** = Register allocation
- **Quantization** = Type lowering

9 Chapter 7: Hands-On Exercises

9.1 Introduction: Learning by Doing

The exercises in this chapter are designed to solidify your understanding of compiler concepts through practical implementation. Each exercise builds upon the concepts we’ve covered, challenging you to apply your knowledge in increasingly sophisticated ways. These aren’t just academic exercises - they mirror real-world compiler engineering challenges you’ll encounter in production systems.

Goals of These Exercises:

- **Reinforce Core Concepts**: Apply IR manipulation, pattern matching, and transformation techniques in practice
- **Develop Compiler Intuition**: Understand trade-offs between different optimization strategies
- **Build Debugging Skills**: Learn to verify correctness and debug transformation passes
- **Connect Theory to Practice**: See how abstract concepts translate to real optimizations

9.2 Exercise 1: Build Your Own Optimizer - Store-Load Forwarding

9.2.1 Context and Motivation

Memory operations are often bottlenecks in program performance. When a program stores a value to memory and immediately loads it back, we're wasting cycles on unnecessary memory traffic. Store-load forwarding is a crucial optimization in modern compilers and CPUs that eliminates these redundant operations.

Real-World Impact: This optimization is so important that modern CPUs implement it in hardware. By doing it in the compiler, we can eliminate the operations entirely, reducing both memory bandwidth and instruction count.

What You'll Learn:

- How to track data flow through memory operations
- Alias analysis basics (when is it safe to forward?)
- The importance of maintaining program correctness during optimization
- How to reason about memory dependencies

```
1 class StoreLoadForwarding(ModulePass):
2     """Forward stored values to subsequent loads
3
4     Transform patterns like:
5         memref.store %value, %mem[%i, %j]
6         %loaded = memref.load %mem[%i, %j]
7     Into:
8         memref.store %value, %mem[%i, %j]
9         // Use %value directly instead of %loaded
10    """
11
12    name = "store-load-forward"
13
14    def apply(self, ctx: Context, module: builtin.ModuleOp) -> None:
15        # TODO: Your implementation here
16        # Step 1: Build a map of recent stores for each memory location
17        # Step 2: For each load, check if we have a recent store
18        # Step 3: Verify indices match exactly (be conservative!)
19        # Step 4: Check for intervening operations that might
20        invalidate
21        # Step 5: Replace load result with stored value:
22        #     load.res.replace_all_uses_with(stored_value)
23        #     load.detach(); load.erase()
24
25        # Hints:
26        # - Start simple: handle only same-block forwarding
27        # - Be conservative: only forward when indices are identical
28        # - Watch for: function calls, other stores to same array
29        # - Remember: correctness > performance
30        pass
31
32    def can_forward(self, store_op, load_op):
33        """Check if it's safe to forward from store to load"""
34        # TODO: Implement safety checks
35        pass
```

Listing 23: Exercise: Store-Load Forwarding

9.2.2 Testing Your Implementation

```
1 def test_store_load_forwarding():
2     """Test your optimization works correctly"""
3
4     # Test case 1: Simple forwarding
5     # store 42 to mem[0]
6     # load from mem[0]  <-- should be replaced with 42
7
8     # Test case 2: Different indices (no forwarding)
9     # store 42 to mem[0]
10    # load from mem[1]  <-- should NOT be forwarded
11
12    # Test case 3: Intervening store
13    # store 42 to mem[0]
14    # store 100 to mem[0]
15    # load from mem[0]  <-- should get 100, not 42
16
17    # Test case 4: Across basic blocks (advanced)
18    # Requires dominance analysis
```

Listing 24: Test Cases for Store-Load Forwarding

9.3 Exercise 2: Create a Custom Dialect - Quantum Computing

9.3.1 Context and Motivation

Quantum computing represents a fundamentally different computational model. By creating a quantum dialect, you'll learn how compiler infrastructure can be extended to support entirely new paradigms. This exercise demonstrates xDSL's extensibility and teaches you to think about computation abstractly.

Why This Matters: As new computational paradigms emerge (quantum, neuromorphic, photonic), the ability to quickly prototype new compiler infrastructure becomes crucial. This exercise teaches you the skills needed to support future computing platforms.

What You'll Learn:

- How to design operations for non-standard computation models
- Type system design for domain-specific constraints
- Verification of domain-specific invariants
- Lowering from high-level domain concepts to implementation

```
1 from xdsl.ir import Dialect, SSAValue, TypeAttribute,
   ParametrizedAttribute
2 from xdsl.irdl import (irdl_op_definition, irdl_attr_definition,
3                        IRDLOperation, operand_def, result_def,
4                        traits_def)
5 from xdsl.traits import Pure
6 from xdsl.dialects.builtin import IntegerType
7
8 # TODO: Define quantum-specific types
9 @irdl_attr_definition
10 class QubitType(ParametrizedAttribute, TypeAttribute):
11     """Type representing a quantum bit"""
12     name = "quantum.qubit"
```

```

13 @irdl_attr_definition
14 class QuantumRegisterType(ParametrizedAttribute, TypeAttribute):
15     """Type for quantum register of n qubits"""
16     name = "quantum.qreg"
17     size: IntAttr
18
19 # TODO: Define quantum gates
20 @irdl_op_definition
21 class HadamardOp(IRDLOperation):
22     """Hadamard gate: creates superposition"""
23     name = "quantum.h"
24
25     input = operand_def(QubitType)
26     result = result_def(QubitType)
27     traits = traits_def(Pure()) # no side effects => DCE may remove
    it
28
29     def __init__(self, inp: SSAValue):
30         super().__init__(operands=[inp], result_types=[QubitType()])
31
32     def verify_(self):
33         """Verify operation is well-formed"""
34         # Hadamard is always valid on a single qubit
35         pass
36
37 @irdl_op_definition
38 class CNOTOp(IRDLOperation):
39     """Controlled-NOT gate"""
40     name = "quantum.cnot"
41
42     control = operand_def(QubitType)
43     target = operand_def(QubitType)
44
45     # CNOT modifies both qubits
46     new_control = result_def(QubitType)
47     new_target = result_def(QubitType)
48
49 @irdl_op_definition
50 class MeasureOp(IRDLOperation):
51     """Measure qubit, collapsing to classical bit"""
52     name = "quantum.measure"
53
54     qubit = operand_def(QubitType)
55     classical_bit = result_def(IntegerType(1))
56
57 QuantumDialect = Dialect("quantum",
58                          [HadamardOp, CNOTOp, MeasureOp],
59                          [QubitType, QuantumRegisterType])
60
61 # TODO: Optimization passes
62 class QuantumCircuitOptimizer(ModulePass):
63     """Optimize quantum circuits"""
64
65     name = "quantum-opt"
66
67     def apply(self, ctx, module):
68         # TODO: Implement optimizations like:
69         # - Gate cancellation (H H = I): match a HadamardOp whose input

```

```

70     is
       #   another HadamardOp, replace its result's uses with the
inner
71     #   input, and erase it.
72     # - Gate commutation
73     # - Circuit depth reduction
74     pass

```

Listing 25: Exercise: Quantum Dialect

9.3.2 Challenge Extensions

- Implement quantum circuit simulation
- Add error correction operations
- Create a cost model for quantum operations
- Implement decomposition to basic gate sets

9.4 Exercise 3: Implement Vectorization

9.4.1 Context and Motivation

Modern processors have SIMD (Single Instruction, Multiple Data) units that can process multiple data elements simultaneously. Vectorization transforms scalar operations to leverage these capabilities, often providing 4-8x speedups. This is a cornerstone optimization in high-performance computing.

Industry Relevance: Every production compiler implements auto-vectorization. ML frameworks rely heavily on vectorized operations for performance. Understanding vectorization is essential for performance engineering.

What You'll Learn:

- How to identify vectorizable patterns
- Dependency analysis for parallel execution
- Handling edge cases (loop remainders, alignment)
- Cost modeling for vectorization decisions

```

1 class Vectorization(ModulePass):
2     """Automatically vectorize loops
3
4     Transform:
5         for i in range(n):
6             c[i] = a[i] + b[i]
7
8     Into:
9         for i in range(0, n-4, 4):
10             c[i:i+4] = a[i:i+4] + b[i:i+4] # Vector operation
11         for i in range(n-n%4, n):
12             c[i] = a[i] + b[i] # Scalar remainder
13     """
14
15     name = "vectorize"
16
17     def __init__(self, vector_width=4):

```

```

18     self.vector_width = vector_width
19
20     def apply(self, ctx: Context, module: builtin.ModuleOp) -> None:
21         for loop in self.find_loops(module):
22             if self.is_vectorizable(loop):
23                 self.vectorize_loop(loop)
24
25     def is_vectorizable(self, loop: scf.ForOp) -> bool:
26         """Check if loop can be vectorized"""
27         # TODO: Check for:
28         # 1. No loop-carried dependencies
29         # 2. Contiguous memory access
30         # 3. Supported operations (add, mul, etc.)
31         # 4. No control flow in loop body
32         pass
33
34     def vectorize_loop(self, loop: scf.ForOp):
35         """Transform loop to use vector operations"""
36         # TODO: Implementation steps:
37         # 1. Create vector loop with stride = vector_width
38         # 2. Replace scalar ops with vector ops
39         # 3. Create remainder loop for n % vector_width iterations
40         # 4. Update memory operations to vector loads/stores
41         pass
42
43     def analyze_dependencies(self, loop: scf.ForOp) -> bool:
44         """Check for loop-carried dependencies"""
45         # TODO: Implement dependency analysis
46         # Look for patterns like a[i] = a[i-1] + b[i]
47         pass

```

Listing 26: Exercise: Auto-Vectorization

9.4.2 Advanced Challenges

- Handle non-unit stride access patterns
- Implement cost model to decide when vectorization is profitable
- Support reduction operations (sum, max)
- Handle conditional operations with masking

9.5 Exercise 4: Build a Simple JIT Compiler

9.5.1 Context and Motivation

Just-In-Time compilation bridges the gap between interpretation and ahead-of-time compilation. By compiling code at runtime, JIT compilers can optimize based on actual execution patterns. This exercise gives you hands-on experience with the full compilation pipeline.

Real-World Applications: JIT compilation powers JavaScript engines (V8), Java VMs (HotSpot), Python (PyPy), and ML frameworks (JAX, PyTorch). Understanding JIT compilation is crucial for modern system design.

What You'll Learn:

- The complete compilation pipeline from source to execution
- Runtime code generation techniques

- Optimization decisions based on runtime information
- Integration between compiled and interpreted code

```

1 import ctypes
2 from typing import Dict, Callable
3
4 class ExpressionJIT:
5     """JIT compile and execute mathematical expressions
6
7     This JIT compiler will:
8     1. Parse expressions into AST
9     2. Generate xDSL IR
10    3. Optimize the IR
11    4. Generate executable code
12    5. Return a callable Python function
13    """
14
15    def __init__(self):
16        self.cache: Dict[str, Callable] = {}
17        self.optimization_level = 2
18
19    def compile(self, expr_string: str) -> Callable:
20        """Compile expression to native function"""
21
22        # Check cache first
23        if expr_string in self.cache:
24            return self.cache[expr_string]
25
26        # Step 1: Parse expression
27        ast = self.parse_expression(expr_string)
28
29        # Step 2: Generate IR
30        ir_module = self.generate_ir(ast)
31
32        # Step 3: Optimize
33        if self.optimization_level > 0:
34            ir_module = self.optimize_ir(ir_module)
35
36        # Step 4: Generate machine code
37        machine_code = self.generate_code(ir_module)
38
39        # Step 5: Create callable wrapper
40        func = self.create_callable(machine_code)
41
42        # Cache for future use
43        self.cache[expr_string] = func
44
45        return func
46
47    def parse_expression(self, expr: str):
48        """Parse expression into AST"""
49        # TODO: Implement expression parser
50        # Support: numbers, variables, +, -, *, /, parentheses
51        # Example: "x * 2 + y" -> BinaryOp('+', BinaryOp('*', Var('x'),
52        Num(2)), Var('y'))
53        pass

```

```

54     def generate_ir(self, ast) -> builtin.ModuleOp:
55         """Generate xDSL IR from AST"""
56         # TODO: Create IR module with function
57         # Function should take dict of variables, return result
58         pass
59
60     def optimize_ir(self, module: builtin.ModuleOp) -> builtin.ModuleOp:
61         """Apply optimization passes"""
62         passes = [
63             ConstantFolding(),
64             CommonSubexpressionElimination(),
65             DeadCodeElimination(),
66         ]
67
68         for opt_pass in passes:
69             opt_pass.apply(module)
70
71         return module
72
73     def generate_code(self, module: builtin.ModuleOp) -> bytes:
74         """Generate machine code or Python bytecode"""
75         # TODO: Options:
76         # 1. Generate Python bytecode using 'compile()'
77         # 2. Use LLVM bindings to generate native code
78         # 3. Generate C code and compile with ctypes
79         pass
80
81     def create_callable(self, code: bytes) -> Callable:
82         """Create Python-callable function from machine code"""
83         # TODO: Wrap machine code in Python-callable interface
84         pass
85
86 # Usage example:
87 jit = ExpressionJIT()
88 f = jit.compile("x * 2 + y")
89 result = f(x=5, y=3) # Should return 13
90
91 # Advanced: Support for conditionals
92 g = jit.compile("x > 0 ? x * 2 : -x")
93 result2 = g(x=-5) # Should return 5

```

Listing 27: Exercise: JIT Compiler

9.5.2 Implementation Hints

```

1 # Simple approach using Python's compile()
2 def generate_python_code(self, module: builtin.ModuleOp) -> str:
3     """Generate Python source from IR"""
4     code = "def compiled_func(**variables):\n"
5     # Walk IR and generate Python code
6     # ...
7     return code
8
9 def create_callable_simple(self, py_code: str) -> Callable:
10    """Create function from Python code"""
11    exec_globals = {}
12    exec(py_code, exec_globals)

```

```
13 return exec_globals['compiled_func']
```

Listing 28: JIT Implementation Starter Code

10 Chapter 8: Real-World Applications

10.1 Case Study 1: Optimizing Neural Network Inference

Let's optimize a real neural network for deployment:

```
1 def optimize_nn_for_deployment(model: NeuralNetwork):
2     """Optimize neural network for inference"""
3
4     # Convert to IR
5     ir = convert_nn_to_ir(model)
6
7     # Apply optimizations
8     optimizations = [
9         # Structural optimizations
10        FuseLayers(),          # Conv+BN, Linear+ReLU
11        RemoveDropout(),      # Not needed for inference
12
13        # Quantization
14        QuantizeWeights(bits=8),
15        QuantizeActivations(bits=8),
16
17        # Memory optimizations
18        InplaceOperations(),   # Reuse buffers
19        MemoryPlanning(),      # Minimize allocation
20
21        # Target-specific
22        UseTargetIntrinsics(), # AVX, NEON, etc.
23        AutoTuneKernels(),     # Platform-specific tuning
24    ]
25
26    for opt in optimizations:
27        ir = opt.apply(ir)
28
29    return ir
```

Listing 29: Neural Network Optimization Pipeline

10.2 Case Study 2: Domain-Specific Language Compiler

Build a compiler for financial computations:

```
1 # Illustrative: invented 'finance' dialect.
2 from xdsl.ir import Dialect
3 from xdsl.irdl import irdl_op_definition, IRDLOperation, operand_def,
4   result_def
5 from xdsl.dialects.builtin import f64
6
7 @irdl_op_definition
8 class BlackScholesOp(IRDLOperation):
9     """Black-Scholes option pricing"""
10    name = "finance.black_scholes"
11
12    spot = operand_def(f64)
```



```

12     strike = operand_def(f64)
13     time = operand_def(f64)
14     rate = operand_def(f64)
15     volatility = operand_def(f64)
16
17     call_price = result_def(f64)
18     put_price = result_def(f64)
19
20 FinanceDialect = Dialect("finance", [BlackScholesOp])
21
22 # Optimize for numerical stability
23 class NumericalStabilityPass(ModulePass):
24     """Ensure numerical stability in financial calculations"""
25
26     name = "numerical-stability"
27
28     def apply(self, ctx, module):
29         for op in module.walk():
30             if isinstance(op, math.ExpOp):
31                 # Prevent overflow/underflow
32                 self.add_range_check(op)
33             elif isinstance(op, arith.DivfOp):
34                 # Prevent division by zero
35                 self.add_zero_check(op)

```

Listing 30: Financial DSL Compiler

10.3 Case Study 3: Compiler for Distributed Computing

Compile programs for distributed execution:

```

1 # Illustrative: invented 'dist' dialect.
2 from xdsl.ir import Dialect
3 from xdsl.irdl import (irdl_op_definition, IRDLOperation,
4                        operand_def, result_def, attr_def)
5 from xdsl.dialects.builtin import TensorType, StringAttr
6
7 @irdl_op_definition
8 class AllReduceOp(IRDLOperation):
9     """All-reduce across distributed nodes"""
10     name = "dist.allreduce"
11
12     input = operand_def(TensorType)
13     output = result_def(TensorType)
14     reduction = attr_def(StringAttr) # "sum", "max", etc.
15
16 DistributedDialect = Dialect("dist", [AllReduceOp])
17
18 class DistributionPass(ModulePass):
19     """Distribute computation across nodes"""
20
21     name = "distribute"
22
23     def apply(self, ctx, module):
24         # Analyze data dependencies
25         deps = self.analyze_dependencies(module)
26
27         # Partition computation

```

```

28     partitions = self.partition_computation(module, deps)
29
30     # Insert communication operations
31     for partition in partitions:
32         self.insert_communication_ops(partition)
33
34     # Generate distributed code
35     return self.generate_distributed_code(partitions)

```

Listing 31: Distributed Computing Compiler

11 Project: Building a Matrix Operations Compiler

11.1 Project Overview

In this project, you'll build a complete compiler for a domain-specific language (DSL) focused on matrix operations. This project demonstrates the full compilation pipeline: from parsing high-level Python-like code, through IR generation and optimization, to generating efficient low-level code. The project is inspired by real-world compilers like those used in machine learning frameworks (TensorFlow, PyTorch) that optimize matrix operations for performance. The code is available at <https://github.com/djtodoro/matrix-toy-lang>.

Why Matrix Operations? Matrix operations are fundamental to scientific computing, machine learning, and graphics. They offer rich optimization opportunities like:

- Algebraic simplifications (e.g., $(A^T)^T = A$)
- Operation fusion (combining multiple operations into one)
- Memory layout optimizations (row-major vs column-major)
- Parallelization opportunities

Learning Objectives:

- Understand the complete compiler pipeline architecture
- Learn how to leverage existing tools (Python AST) with custom IR
- Implement pattern-matching optimizations
- Experience the design decisions in creating a custom dialect
- Practice incremental development and testing

Project Structure: The compiler consists of five main phases:

1. **Frontend:** Parse Python code using the `ast` module
2. **Dialect Design:** Create a custom matrix operations dialect
3. **IR Generation:** Convert Python AST to dialect operations
4. **Optimization:** Apply pattern-based transformations
5. **Lowering:** Generate executable code (optional)

11.2 Phase 1: Frontend - Parsing the Input

The compiler begins by parsing the input program. For this project, we'll use Python as our source language and leverage Python's built-in `ast` module to parse the code.

Example Input Program:

```
1 def matrix_computation(A, B):
2     C = A @ B           # Matrix multiplication
3     D = C.T.T           # Double transpose (can be optimized to C)
4     E = D * 2.0         # Scalar multiplication
5     F = E.T             # Single transpose
6     G = F.T             # Another transpose (F.T.T = E)
7     H = G + B.T         # Matrix addition
8     result = H.T.T      # Final double transpose
9     return result
```

Listing 32: Example matrix operations with optimization opportunities

This example contains several optimization opportunities:

- Three instances of double transpose that can be eliminated
- Potential for operation fusion
- Opportunities for common subexpression elimination

11.3 Phase 2: Building the Parser

The parser traverses the Python AST and extracts matrix operations. We use the Visitor pattern to walk the AST and identify relevant operations.

Key Parser Tasks:

1. Identify matrix operations (matmul, transpose, add, scalar mul)
2. Track variable definitions and uses
3. Build a sequence of operations for IR generation
4. Detect optimization patterns (like double transpose)

Parser Architecture:

```
1 import ast
2
3 class MatrixOperationExtractor(ast.NodeVisitor):
4     def __init__(self):
5         self.operations = [] # List of extracted operations
6         self.variables = {}  # Variable tracking
7
8     def visit_Assign(self, node):
9         # Extract the operation from the right-hand side
10        op_info = self.analyze_expression(node.value)
11        target = node.targets[0].id
12
13        self.operations.append({
14            'target': target,
15            'operation': op_info
16        })
17
18    def analyze_expression(self, node):
19        # Pattern matching for different operations
```

```

20     if isinstance(node, ast.BinOp):
21         if isinstance(node.op, ast.MatMult): # @ operator
22             return {'op': 'matmul', ...}
23         elif isinstance(node.op, ast.Mult): # * operator
24             return {'op': 'scalar_mul', ...}
25     elif isinstance(node, ast.Attribute):
26         if node.attr == 'T': # Transpose
27             # Check for double transpose pattern
28             base = self.analyze_expression(node.value)
29             if base.get('op') == 'transpose':
30                 return {'op': 'double_transpose', ...}
31             return {'op': 'transpose', ...}
32     # ... handle other operations

```

Listing 33: Parser structure and key methods

Key Insights:

- The parser performs early pattern detection (e.g., double transpose)
- Operations are stored as a linear sequence for easy IR generation
- Variable tracking enables use-def chain analysis

11.4 Phase 3: Designing the Matrix Dialect

A dialect in xDSL/MLIR is a collection of operations and types that represent concepts at a specific abstraction level. For matrix operations, we need to design operations that:

- Capture the semantics of matrix operations precisely
- Enable verification of correctness (e.g., dimension checking)
- Facilitate pattern matching for optimizations
- Support gradual lowering to hardware operations

Core Operations in the Matrix Dialect:

```

1 from xdsl.ir import Dialect, SSAValue
2 from xdsl.irdl import (irdl_op_definition, IRDLOperation,
3                        operand_def, result_def, traits_def)
4 from xdsl.traits import Pure
5 from xdsl.dialects.builtin import TensorType
6 from xdsl.utils.exceptions import VerifyException
7
8 @irdl_op_definition
9 class MatMulOp(IRDLOperation):
10     """Matrix multiplication: C = A @ B"""
11     name = "matrix.matmul"
12
13     lhs = operand_def(TensorType)
14     rhs = operand_def(TensorType)
15     result = result_def(TensorType)
16
17     # Shape inference: (m x n) @ (n x p) -> (m x p)
18     def verify_(self):
19         # Check dimension compatibility
20         if self.lhs.type.get_shape()[-1] != self.rhs.type.get_shape()
[0]:

```

```

21         raise VerifyException("incompatible matmul dimensions")
22
23 @irdl_op_definition
24 class TransposeOp(IRDLOperation):
25     """Matrix transpose: B = AT"""
26     name = "matrix.transpose"
27
28     input = operand_def(TensorType)
29     result = result_def(TensorType)
30     traits = traits_def(Pure()) # pure => DCE may remove dead
    transposes
31
32     # Shape transformation: (m x n) -> (n x m)
33     # Key for optimization patterns
34     def __init__(self, inp: x
35         \item Create a function wrapper for the operations
36         \item Map Python variables to SSA values
37         \item Generate dialect operations in sequence
38         \item Maintain use-def chains through SSA form
39 \end{enumerate}
40
41 \textbf{Example IR Generation:}
42 \begin{lstlisting}[language=Python, caption=Translating parsed
    operations to IR]
43 class MatrixIRGenerator:
44     def __init__(self):
45         self.var_map = {} # Maps Python variables to SSA values
46
47     def generate_operation(self, op_info):
48         op_type = op_info['operation']['op']
49         target = op_info['target']
50
51         if op_type == 'matmul':
52             # C = A @ B becomes:
53             # %c = matrix.matmul %a, %b : (f32, f32) -> f32
54             left = self.var_map[op_info['operation']['left']]
55             right = self.var_map[op_info['operation']['right']]
56             result = MatMulOp(left, right).result
57             self.var_map[target] = result
58
59         elif op_type == 'double_transpose':
60             # D = C.T.T becomes (before optimization):
61             # %tmp = matrix.transpose %c : (m x n) -> (n x m)
62             # %d = matrix.transpose %tmp : (n x m) -> (m x n)
63             matrix = self.var_map[op_info['operation']['matrix']]
64             temp = TransposeOp(matrix).result
65             result = TransposeOp(temp).result
66             self.var_map[target] = result

```

Listing 34: Key dialect operations and their design

Generated IR Example:

```

1 func @matrix_computation(%A: matrix<2x3xf32>, %B: matrix<3x2xf32>)
2     -> matrix<2x2xf32> {
3     %C = matrix.matmul %A, %B : (matrix<2x3xf32>, matrix<3x2xf32>)
4         -> matrix<2x2xf32>
5     %tmp1 = matrix.transpose %C : matrix<2x2xf32> -> matrix<2x2xf32>
6     %D = matrix.transpose %tmp1 : matrix<2x2xf32> -> matrix<2x2xf32>
7     %E = matrix.scalar_mul %D, 2.0 : matrix<2x2xf32> -> matrix<2x2xf32>

```

```

8 %F = matrix.transpose %E : matrix<2x2xf32> -> matrix<2x2xf32>
9 %G = matrix.transpose %F : matrix<2x2xf32> -> matrix<2x2xf32>
10 %B_t = matrix.transpose %B : matrix<3x2xf32> -> matrix<2x3xf32>
11 %H = matrix.add %G, %B_t : (matrix<2x2xf32>, matrix<2x3xf32>)
12     -> matrix<2x2xf32>
13 %tmp2 = matrix.transpose %H : matrix<2x2xf32> -> matrix<2x2xf32>
14 %result = matrix.transpose %tmp2 : matrix<2x2xf32> -> matrix<2x2xf32>
15 return %result : matrix<2x2xf32>
16 }

```

Listing 35: Matrix dialect IR before optimization

11.5 Phase 5: Optimization - Double Transpose Elimination

The optimization phase applies pattern-based transformations to improve the IR. Our primary optimization eliminates the mathematical identity $(A^T)^T = A$.

Pattern Matching Approach:

```

1 from xdsl.pattern_rewriter import (
2     PatternRewriter, RewritePattern, op_type_rewrite_pattern,
3 )
4
5 class DoubleTransposeElimination(RewritePattern):
6     """Eliminate pattern: transpose(transpose(X)) = X"""
7
8     eliminated_count = 0
9
10    @op_type_rewrite_pattern
11    def match_and_rewrite(self, op: TransposeOp, rewriter:
12    PatternRewriter):
13        # Check if input is also a transpose
14        if isinstance(op.input.owner, TransposeOp):
15            # Found the pattern!
16            original = op.input.owner.input
17
18            # Replace double transpose with the original value
19            rewriter.replace_all_uses_with(op.result, original)
20            rewriter.erase(op)
21
22            # Statistics tracking
23            self.eliminated_count += 1

```

Listing 36: Pattern-based optimization using xDSL's rewriter

How the Optimization Works:

1. **Pattern Detection:** Walk the IR looking for transpose operations
2. **Match Checking:** Check if the input to a transpose is another transpose
3. **Replacement:** Replace the double transpose with the original matrix
4. **Cleanup:** Remove dead code (unused intermediate transposes)

Before and After Optimization:

```

1 // Before optimization:
2 %tmp1 = matrix.transpose %C : matrix<2x2xf32> -> matrix<2x2xf32>
3 %D = matrix.transpose %tmp1 : matrix<2x2xf32> -> matrix<2x2xf32>
4

```

```

5 // After optimization:
6 %D = %C // Direct use, no transposes needed

```

Listing 37: IR transformation example

Additional Optimization Opportunities:

- **Algebraic Simplifications:** $(A + B)^T = A^T + B^T$
- **Operation Fusion:** Combine scalar_mul operations
- **Common Subexpression Elimination:** Reuse computed transposes
- **Matrix Chain Optimization:** Optimal parenthesization for multiple multiplications

11.6 Phase 6: The Complete Compiler Pipeline

The complete compiler integrates all phases into a cohesive pipeline. Here's how the components work together:

Pipeline Architecture:

```

1 class MatrixCompiler:
2     def compile(self, source_code: str) -> str:
3         # Phase 1: Parse Python to AST
4         ast_tree = ast.parse(source_code)
5
6         # Phase 2: Extract matrix operations
7         operations = MatrixOperationExtractor().extract(ast_tree)
8
9         # Phase 3: Generate IR
10        module = MatrixIRGenerator().generate(operations)
11
12        # Phase 4: Optimize
13        optimizer = MatrixOptimizationPipeline()
14        optimizer.apply(module)
15
16        # Phase 5: Lower to target (optional)
17        # Could lower to LLVM, C, or other targets
18
19        return str(module)

```

Listing 38: Main compiler driver structure

Running the Compiler:

```

1 # Compile and optimize
2 python matrix_compiler.py input.py --optimize
3
4 # View IR at different stages
5 python matrix_compiler.py input.py --emit-ir
6 python matrix_compiler.py input.py --emit-optimized-ir
7
8 # Generate executable (if lowering is implemented)
9 python matrix_compiler.py input.py -o output

```

Listing 39: Command-line usage

Example Compilation Flow:

1. **Input:** Python code with matrix operations
2. **Parse:** Extract operation sequence

3. **Generate:** Create matrix dialect IR
4. **Optimize:** Apply transformations (3 double transposes eliminated)
5. **Output:** Optimized IR or lowered code

11.7 Testing and Validation

A robust compiler needs comprehensive testing at each phase:

Testing Strategy:

1. **Unit Tests:** Test each phase independently
2. **Integration Tests:** Test the complete pipeline
3. **Correctness Tests:** Verify semantic preservation
4. **Performance Tests:** Measure optimization effectiveness

But, we have not added tests yet... :)

11.8 Project Extensions and Exercises

Recommended Extensions:

1. Additional Optimizations:

- Implement $(A \cdot B)^T = B^T \cdot A^T$ transformation
- Add common subexpression elimination
- Implement operation fusion (combine multiple scalar multiplications)

2. Advanced Features:

- Add support for matrix inverse operations
- Implement shape inference for unknown dimensions
- Add support for batched operations

3. Code Generation:

- Lower to LLVM IR for execution
- Generate optimized C code
- Target GPU operations (CUDA/OpenCL)

4. Analysis Passes:

- Add cost model for operations
- Implement data dependency analysis
- Add memory usage optimization

Project Structure:


```

1 matrix-compiler/
2 |-- src/
3 |   |-- frontend/
4 |   |   |--- parser.py           # AST parsing
5 |   |   |-- lexer.py            # Tokenization (if needed)
6 |   |-- dialect/
7 |   |   |-- matrix_ops.py       # Operation definitions
8 |   |   |-- types.py           # Type definitions
9 |   |-- transforms/
10 |   |   |-- optimize.py        # Optimization passes
11 |   |   |-- lower.py           # Lowering passes
12 |   |-- compiler.py            # Main driver
13 |-- tests/
14 |   |-- test_parser.py         # Parser tests
15 |   |-- test_dialect.py       # Dialect tests
16 |   |-- test_optimize.py      # Optimization tests
17 |-- examples/
18 |   |-- simple.py             # Simple examples
19 |   |-- complex.py            # Complex examples
20 |-- README.md                 # Documentation

```

Listing 40: Recommended project organization

11.9 Learning Outcomes and Key Takeaways

By completing this project, students will have gained practical experience with:

Core Compiler Concepts:

- **Frontend Design:** Parsing and semantic analysis
- **IR Design:** Creating domain-specific abstractions
- **Pattern Matching:** Implementing optimization patterns
- **SSA Form:** Understanding use-def chains
- **Verification:** Ensuring correctness at each phase

Software Engineering Skills:

- Designing modular, testable compiler components
- Writing comprehensive test suites
- Documenting complex software systems
- Using modern compiler infrastructure (xDSL/MLIR)

Mathematical Foundations:

- Matrix algebra and its optimization opportunities
- Correctness preservation through transformations
- Cost models and performance analysis

11.10 Conclusion

This project demonstrates the complete journey of building a domain-specific compiler. Starting from high-level Python code, we’ve created a custom IR dialect, implemented pattern-based optimizations, and seen how modern compiler infrastructure enables rapid development of specialized compilers.

The matrix operations compiler serves as a foundation for understanding larger systems like TensorFlow’s XLA, PyTorch’s TorchScript, and other domain-specific compilers that power modern machine learning and scientific computing.

Next Steps:

- Explore the xDSL/MLIR ecosystem for more advanced dialects
- Study production compilers like LLVM, GCC, or specialized ML compilers
- Implement more sophisticated optimizations (auto-vectorization, parallelization)
- Build compilers for other domains (graphics, cryptography, databases)

Remember: Compilers are not just about code generation—they’re about bridging the gap between human intent and machine execution, enabling both productivity and performance.